

Physics-based modeling of crowd evacuation in the Unity game engine

Walter Alan Cantrell*, Mikel D. Petty[†],
Samantha L. Knight[‡] and Whitney K. Schueler[§]

*University of Alabama in Huntsville
301 Sparkman Drive, OKT N353, Huntsville
Alabama 35899, United States of America*

**wac0010@uah.edu*

†pettym@uah.edu

‡slk0002@uah.edu

§knightw@uah.edu

Received 20 December 2016

Accepted 5 February 2018

Published 14 March 2018

Crowds of people are often found in enclosed or constricted spaces. Evacuation in such situations is usually conducted calmly, but real or perceived danger may trigger panic. In panicked crowds, the interpersonal distance crowd members normally observe is often overwhelmed by the physical pressure of crowd members pushing against each other. That pressure can both slow evacuation and lead to injury or death. Models have been developed to study crowd evacuation in a range of situations. This paper describes the implementation, testing, and validation of a crowd evacuation model using Unity, a commercial computer game engine. A realistic physics-based model of crowd movement that calculates and considers the physical pressure crowd members exert on each other was implemented in Unity. The implemented model was tested under both nonpanicked and panicked scenarios; those tests exhibited known qualitative characteristics of such scenarios. The model was then quantitatively validated by comparing its results to the outcome of an actual evacuation event, the 2003 fire at The Station nightclub in Rhode Island. The implementation and validation show that Unity can be an effective tool for evacuation modeling.

Keywords: Crowd modeling; crowd movement; evacuation modeling; game engines; Unity.

1. Introduction and Motivation

In modern society, crowds of people are often gathered together in enclosed or constricted spaces. This can occur in many situations, including sporting events, musical concerts, political rallies, and religious observances. Although the crowds in these situations usually remain calm, panic within a crowd can be triggered by

*Corresponding author.

both real and perceived threats, such as the sight of fire, the smell of pepper spray, or even simply the sight of someone running. In panicked situations, evacuation of a crowd from a confined space can be dangerous and potentially deadly. Panicked crowd members may stampede or press on each other with enough force to result in injuries or deaths due to trampling or crushing. On numerous occasions crowds have panicked, sometimes without reason, leading to tragic outcomes. A chronicle of such events, covering some 150 years, is available.¹

This paper's intended scope is not as broad as crowd movement in general or even crowd evacuation in particular. Rather, it is primarily focused on two contributions. First, the mechanics of implementing a physics-based crowd movement model in a computer game engine, specifically Unity, as opposed to doing so using a simulation platform or a general purpose programming language, are described in some detail. The intent is to enable crowd modeling researchers to use game engines as environments within which crowd movement models can be developed and tested, and to assist computer game developers to implement physically realistic models of crowd movement. To that end, the Unity-based implementation is presented in this paper in sufficient detail to facilitate similar implementations. Second, a quantitative validation of the implemented model based on an actual crowd evacuation event is reported to show that such a model can be realistically accurate and to demonstrate the validation process.

The Unity game engine was chosen as the implementation environment because of the relative ease of creating a three-dimensional (3D) realistic representation of real or imagined worlds when compared to other environments or high level programming languages. Unity is a cross platform game engine often used to develop video games. Game engines typically contain a rendering engine for 2D or 3D graphics generation, and a physics engine, which includes collision detection functionality. These attributes are particularly useful for developing a crowd model. Unity is also available to students at no cost and numerous assets are available for download.

This paper is organized as follows. Section 2 presents a brief review of related work. Section 3 explains the physics-based crowd movement model. Section 4 details the implementation of the crowd movement model and the virtual world within which crowd movement is simulated using the Unity game engine, including the differences between nonpanicked and panicked evacuation. This section also describes the qualitative functionality testing of the implemented model. Section 5 reports the quantitative validation of the implemented model using an actual crowd evacuation event. Finally, Sec. 6 reports the paper's findings and recommendations for future work.

2. Related Work

The research literature on the subject of crowd modeling is wide-ranging and large, and a comprehensive survey is beyond the scope of this paper. Instead, representative examples selected with the intent of placing the subject of this paper,

implementation of a realistic crowd movement model in a game engine, in the context of the larger body of crowd modeling research are presented. More extensive surveys are recommended to readers with broader interests.^{2,3}

It is useful to distinguish between models of crowd behavior and models of crowd movement. Crowd behavior models replicate the mental decisions made by crowd members about what actions to perform. The repertoire of actions available to crowd members typically depends on the intended uses of the model. They may be limited to movement for an evacuation model, or they may include nonmovement actions, such as chanting, throwing a stone, or waving a banner in a model of protests and riots. Crowd behavior models may use any of a range of modeling methods, including various approaches from artificial intelligence (AI). Personal, cultural, and situational factors can affect the behavior of crowd members.⁴ For example, an empirical study of the observed range and frequency of specific crowd member actions and their relation to crowd member mental states was used as the basis for a crowd behavior model implemented with game technology.⁵

Crowd movement, in contrast, is a specific type among many of crowd behavior. Once a crowd behavior model, a script, or some other method has determined that a crowd member should move to a new location, the crowd movement model should execute the movement. The pre-movement planning of a route from a crowd member's starting location to his/her intended destination can be included. Crowd movement models replicate the physical movement of crowd members from one location to another. Crowd movement models may be based on physics or kinematics. Realistic movement is important because human observers are quite sensitive to even slightly unnatural behavior in simulated humans.⁶ Crowd movement models differ from individual movement models in that crowd models should consider the close proximity and resulting congestion that characterizes movement in a crowd. Terrain, weather, and other factors may also affect the movement.

Reynolds' well-known model of the motion of a flock of abstracted birds (which he called "boids") was an important impetus toward crowd movement modeling. That model used ideas from particle systems and represented each individual bird as a particle.⁷ Similarly, a particle system-based model was used more recently for fire evacuation studies.⁸ Cellular automata were applied to represent crowd member navigation.⁹ A model of groups moving in close proximity at speeds fast enough that their movement dynamics (acceleration, velocity, and turning radius) were important gave special attention to collision avoidance.¹⁰ A crowd member movement algorithm developed specifically for high-density crowds calculated a force vector (direction) for a crowd member's movement as a weighted sum of contributing force vectors representing the destination, walls, obstacles, other crowd members, and directional hysteresis.¹¹

Crowd movement models are often closely linked to the underlying terrain representation. A model of crowd movement for use in a simulation of urban emergency search and rescue operations depended upon a pre-execution discretization of the terrain area into a graph of nodes and edges.¹² Crowd members probabilistically

moved from node to node along edges based on a calculated attraction value for each node that considered psychological and environment factors and potentially changed with each time step. Another model, operating on terrain discretized into cells, combined a force-based mechanism and the D* incremental graph search algorithm to control the movement of crowd members.¹³

Evacuation is an application of special interest in crowd movement modeling. In evacuation, crowd movement is complicated by sociological or emotional elements,^{14,15} especially panic.¹⁶ Panicked evacuees tend to focus on immediate personal interests and ignore social and cultural norms, such as interpersonal distance, that would otherwise constrain their movement.¹⁷ Panic may reduce crowd members' attention, causing closer exits to be overlooked, and consequently panic may actually delay the escape of crowd members in evacuations.¹⁸ Crowd movement models have been developed for a variety of evacuation-related applications, including evaluating the effect of obstacles on evacuation efficiency,¹⁹ analyzing evacuation during building fires,²⁰ evacuation during terrorist bomb attacks,²¹ analyzing evacuation of subway systems using escalators,²² analyzing evacuation of road tunnels,²³ and building design features to facilitate evacuation.^{15,24} A cellular discrete event simulation environment that had earlier been used to model nonevacuation crowd movement²⁵ was subsequently used to implement a suite of evacuation models that represented different aspects of crowd member behavior during emergencies.²⁶ The effect of adverse sight conditions, as might be found in a smoke-filled building, were modeled using blind following movement rules.²⁷ Biologically inspired models based on insect swarms have also been used to model crowd panic responses.^{28,29} To be realistic, models of crowd movement, especially panicked movement, should allow for the possibility of injury or death due to pressure and trampling. Such injuries are of substantial concern in evacuation, and crowd models have been used to specifically analyze this risk.³⁰

Some instances of crowd model implementation in game engines exist. In addition to the model mentioned earlier,⁵ crowd members have been integrated into Unreal Tournament 2004³¹ and Gamestudio A7.¹⁵ The work described here differs from some of the earlier projects in demonstrating the integration of a physics-based crowd movement model into a game engine (Unity). In a closely related effort, the same physical model of crowd movement used in this work was implemented in a different game engine (Unreal Tournament).³²

3. Crowd Movement Model

The physics-based crowd movement model used in this work is due to Helbing, Farkas, and Vicsek.³³ (For brevity, hereinafter it will be referred to simply as the Helbing model). It is based on the inclusion of social forces in a Newtonian balance of forces model. Recall that in the Newtonian model of force and motion, $F = ma$, where F is the force on an object, m is the mass of the object, and the acceleration a is the rate of change of velocity at a point in time. The forces acting on a person

i in a crowd can be written as Eq. (1), where m_i is the mass of the person and $a = (d\vec{v}_i/dt)$

$$m_i \frac{d\vec{v}_i}{dt} = m_i \frac{v_i^0(t) \vec{e}_i^0 - \vec{v}_i(t)}{\tau_i} + \sum_{j \neq i} \vec{f}_{i,j} + \sum_W \vec{f}_{i,W}, \quad (1)$$

The first term on the right-hand side of Eq. (1) is related to the motion of person i over an interval of time τ_i , where the person's initial speed is v_i^0 and desired direction is $\vec{e}_i^0$.

From the definition of the derivative, we know that $d\vec{v}_i/dt = \lim_{\Delta t \rightarrow 0} (\Delta \vec{v}/\Delta t)$. The quantity $(v_i^0(t) \vec{e}_i^0 - \vec{v}_i(t))/\tau_i$ represents the difference between a person's current speed and direction, $\vec{v}_i(t)$, and a person's desired speed and direction $v_i^0(t) \vec{e}_i^0$, over a small interval of time, τ_i . Thus the product $m_i(v_i^0(t) \vec{e}_i^0 - \vec{v}_i(t))/\tau_i$ represents a discrete approximation of the force due to the difference in desired motion and current motion. The terms $\sum_{j \neq i} \vec{f}_{i,j}$ and $\sum_W \vec{f}_{i,W}$ in Eq. (1) represent the forces resulting from interactions with other persons in the crowd and with walls and other obstacles, respectively.

For each interacting pair of persons in the crowd i and j ,

$$\vec{f}_{ij} = \{A_i e^{\frac{r_{ij} - d_{ij}}{B_i}} + kg(r_{ij} - d_{ij})\} \vec{n}_{ij} + \kappa g(r_{ij} - d_{ij}) \Delta v_{ji}^t \vec{t}_{ij}. \quad (2)$$

The quantity $r_{ij} - d_{ij}$ occurs three times in Eq. (2); r_{ij} is the sum of the radii of each person, i and j ; and d_{ij} is the distance from the center of i to the center of j . If this quantity is positive, the persons are in contact with each other, or have collided.

In the first product of the right-hand side of Eq. (2), the exponential is altered by the constants A_i and B_i . These constants only relate to person i and are independent of person j . A_i and B_i represent person i 's social deportment toward being close to or in contact with another person in a crowd. Obviously, when A_i is large, or if B_i is small, the value of the first product in Eq. (2) becomes more significant. As the value $r_{ij} - d_{ij}$ increases (meaning persons i and j are getting closer), the exponential gets larger, and grows rapidly once $r_{ij} - d_{ij}$ is positive (in a state of collision). Function $g(r_{ij} - d_{ij})$ is equal to the argument when the argument is positive and zero otherwise. The second term in the sum above, $kg(r_{ij} - d_{ij})$ represents the force of an actual collision. This implementation of the Helbing model represents all collisions between persons with the same magnitude of force. This abstraction does not appear to significantly impact fidelity of the model for purposes of this simulation. In this implementation, the model is further simplified to assume that all people are equally averse to physical contact with one another.

The sum $A_i e^{[(r_{ij} - d_{ij})/B_i]} + kg(r_{ij} - d_{ij})$ is multiplied by the vector $\vec{n}_{ij}$, which is a normalized vector that points directly from the center of person j to the center of person i , so the forces related to social deportment and collisions are additive.

The third term in Eq. (2), $\kappa g(r_{ij} - d_{ij}) \Delta v_{ji}^t \vec{t}_{ij}$, is related to the friction between persons i and j . κ is another scaling constant, and the function $g(r_{ij} - d_{ij})$ is as

previously described. Vector $\vec{t}_{ij}$ represents the tangential direction between j and i , and is formed by taking the negative of the z -component and the x -component of $\vec{n}_{ij}$, to form a new vector $\vec{t}_{ij} = (-z, y, x)$. Vector $\vec{t}_{ij}$ ensures that the direction of the force of friction is tangential to the points of contact between the person i and the person j . When in contact with another person or object, the force of friction acts to restrict motion, and is in a direction opposite of motion, tangential to the point of contact between the persons or person and object. The quantity Δv_{ji}^t is a scalar formed by taking the difference between the direction vectors of person i and person j , and taking the dot product of that difference with the vector $\vec{t}_{ij}$. Δv_{ji}^t acts as a multiplier, also scaling the force of friction between colliding persons.

The term $\sum_W \vec{f}_{i,W}$ in Eq. (1) represents the sum of forces a person i is acted on related to walls or other obstacles, consisting of terms such as

$$\vec{f}_{iW} = \{A_i e^{\frac{r_i - d_{iW}}{B_i}} + kg(r_i - d_{iW})\} \vec{n}_{ij} + \kappa g(r_i - d_{iW})(\vec{v}_i \cdot \vec{t}_{iW}) \vec{t}_{iW}. \quad (3)$$

In the calculation of forces $\vec{f}_{iW}$, all terms are similar to the case where interaction involves forces with other persons except for the term involving friction; the terms are similar in that $r_i - d_{iW}$ represents the distance from person i to wall W function g , constants k and κ are analogous. $\vec{t}_{iW}$ is also similar in that it represents the vector tangent to the point of contact between the person and the object.

Because the wall is not moving, the friction between the person and the object is calculated differently, with the factor $\vec{t}_{iW}$ similar to $\vec{t}_{ij}$. Because the object is stationary, the dot product to the person's velocity and tangent vector $\vec{t}_{iW}$ replaces Δv_{ji}^t from Eq. (2). Recall that Δv_{ji}^t includes components of the velocities of persons i and j . In the case of collisions with objects, the objects are stationary, so the scalar Δv_{ji}^t is replaced with $\vec{v}_i \cdot \vec{t}_{iW}$ in the friction term. The sums of these interaction forces, both physical and social, modify the direction that a person is moving in the crowd.³³ Calibrating the model requires appropriate balance between the values of A_i , B_i , k , and κ .

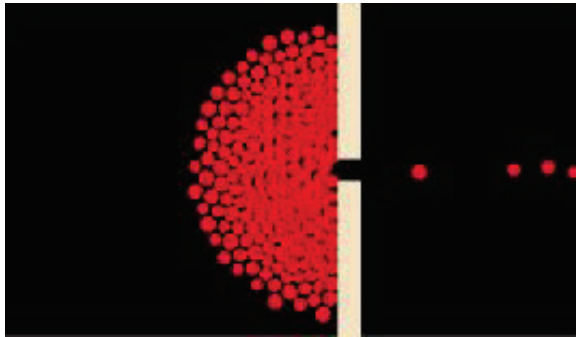


Fig. 1. An example of an arch formation produced by the Helbing evacuation model.³³

Evacuation simulations using the Helbing model typically result in configurations of crowd members similar to the one shown in Fig. 1.¹⁶ An evacuating crowd has collected around a single exit in a semicircular cluster, usually called an “arch”. In a nonpanicked state, the crowd members observe social interpersonal distance constraints and exit safely. In a panicked state, the crowd members may push and shove each other, and the forces related to desired direction and speed dominate the social forces that encourage a member of the crowd to avoid contact.

Consequently, the crowd members are pressed together and into the walls and other obstacles.³³

4. Implementation

This section details the implementation of the crowd movement model and the virtual world within which crowd movement is simulated using the Unity game engine. The differences between the nonpanicked and panicked evacuation scenarios are highlighted. This section also describes the functionality testing of the implemented model.

The initial implementation used a simple evacuation scenario, conveniently used as a baseline for studies on evacuation. (A more realistic scenario, which was used for validation, is described in Sec. 5.) In the simple scenario, a group of crowd members occupying a single rectangular room evacuate through a single exit. The crowd members were implemented so that they could be spawned throughout the room at the outset of scenario execution. Once spawned, the crowd members moved through the exit to a location outside of the room. The crowd movement model included both the nonpanicked and the panicked versions of the Helbing model; the latter included the additional crowd force term, as described in Sec. 3.

4.1. Unity scene

The basic evacuation scenario (room, exit, crowd members, and crowd state, either nonpanicked or panicked) was implemented as a Unity scene. Beginning with an empty scene, the ground was created by adding a plane and positioning it so that the center of the longer side was placed at the origin of the scene. The walls were then constructed by transforming four cube objects and placing them around the plane. The ground and walls were set as collider objects. A collider object in Unity makes the object solid so that other objects cannot pass through it. Visual realism was added using material features available in Unity, which can be applied to the surfaces of objects within an environment. A basic earth material was used for the ground and a brick veneer material for the walls. Clouds were added to a skybox surrounding the scene.

Figure 2 shows an overhead view of the room implemented for the crowd simulation environment. The room was 100 units wide by 50 units across and the walls are 8 units high. The physical world within Unity is mapped using the standard 3D Cartesian coordinates, denoted (x, y, z) , where x and z represent the plane

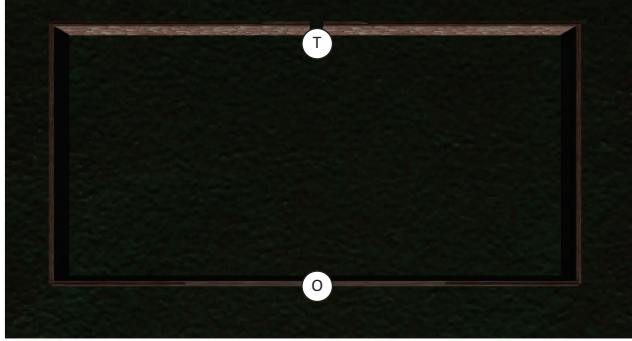


Fig. 2. Overhead view of the Unity scene for the simple evacuation scenario.

containing the ground and y is the up-down dimension. The point labeled O in the figure is the origin $(0, 0, 0)$ of the coordinate system within the scene. A two-unit-wide opening in the center of the wall opposite the origin was the exit that the crowd members used to evacuate the room. This target exit point is labeled T in the figure, and is located at $(0, 0, -49)$.

In order for objects to interact within a scene as they would in real life, colliders must be attached as child objects to the parent objects in the scene. The use and behavior of collider objects is fundamental to physics modeling in the Unity game engine. Colliders cause the game engine to detect collisions between their parent objects. Stationary objects, such as walls, rocks, trees, and even the ground plane must all be parents of collider objects. Without collider objects, objects in the scene would fall through the ground or pass through each other without effect, producing unrealistic behavior. The ground plane and walls each have RigidBody colliders added as child objects. In Unity, a RigidBody collider object has event handling methods that can be called to take specific actions when collisions occur.³⁴

4.2. Crowd members

To implement the crowd members, a Unity asset called the Relaxed Man Character developed by Vu Studios was downloaded from the Unity Asset Store.³⁵ This asset is an example of a Unity prefab object that has been created as a template for instantiation in various games or projects. The Relaxed Man prefab provides a number of characteristics needed for a crowd member, including a fully rigged human figure. Relaxed Man offers three animation options: Walk 1, Walk 2, and Run.³⁵ Walk 1 was modified for use in the nonpanicked scenario and Run was used in the panicked scenario. The two walking animations did not provide the desired smooth motion, necessitating some modification of the animation parameters. The Relaxed Man default physical characteristics were also modified, including modifications to the mass of the character and the size and diameter of the child capsule. Once these modifications had been tested, the modified character was saved as a

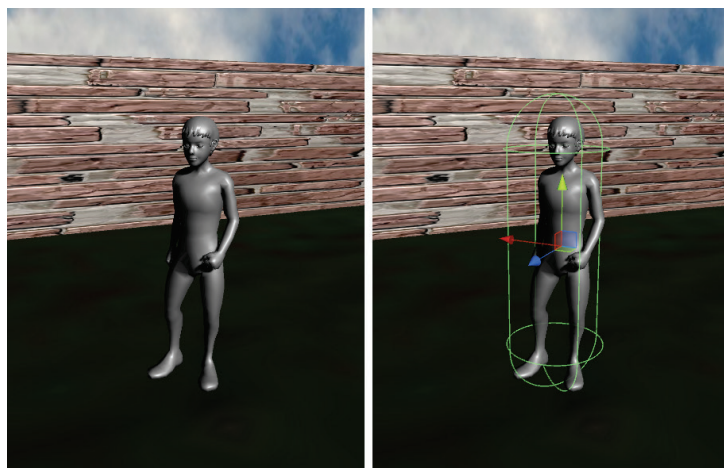


Fig. 3. (Left) Crowd Guy instance inside the simple evacuation scene. (Right) Crowd Guy instance with Capsule Collider.

custom prefab. It could then be instantiated at multiple locations within the scene to create a crowd. The resulting customized version is referred to as Crowd Guy. Both Relaxed Man and Crowd Guy are object classes. Figure 3(left) shows an instance of Crowd Guy in the simple evacuation scenario scene.

Although the static objects in the scene, such as the walls and ground, use the physics-based RigidBody controller, it is preferable for humanoid objects to use another type of controller, called the CharacterController. The benefit of a CharacterController over a RigidBody controller is in the management of the movement of a Crowd Guy instance. Normal motion and animation, including climbing over objects, can be implemented with more natural movement in a CharacterController. In general, the CharacterController is the recommended method of implementing a player or a humanoid nonplayer character in Unity.³⁴

When a CharacterController is created, a Capsule Collider is created by default and attached to the CharacterController. A Capsule Collider is a capsule-shaped collider object that can be used for humanoid assets, in this case the crowd members.³⁴ Figure 3(right) illustrates a Crowd Guy with a CharacterController, showing the Capsule Collider component. The figure also shows a cube adjacent to the abdomen of the Crowd Guy with three arrows emanating from the cube. This cube is centered at the (0, 0, 0) point relative to the Crowd Guy's origin. A set of child data elements, known as the transform, orient the character in the scene. As explained earlier, there is a point within the scene that is the scene's origin. Each Crowd Guy in the scene has a location with x , y , and z components relative to the origin of the scene and a rotation that allows the character to be oriented in the correct direction within the scene.

The Capsule Collider implemented with the CharacterController is preferred to a meshed collider, for performance reasons. While it is possible, and possibly more

realistic from the perspective of viewing a collision, to implement a collider based on the fully meshed humanoid character, the computing overhead is tremendous. Furthermore, the realism of the model is not significantly impacted by the use of the Capsule Collider as opposed to a mesh collider.

In the nonpanicked scenario, the CharacterController's motion is managed by the Unity AI engine, through the extension of the CharacterController with a NavMesh agent. The NavMesh agent facilitates the Crowd Guy movement through the scene without colliding with objects or other Crowd Guys. This functionality is the purpose of the NavMesh agent. The NavMesh agent is discussed further in Sec. 4.3.1. In the panicked scenario, the NavMesh agent is not used with the Crowd Guy, and the Helbing model is implemented through custom scripting.

Multiple instances, or "clones" in Unity terms, of the Crowd Guy object class are instantiated within the room to create the crowd. The simulation was implemented to be run with varying crowd sizes. An initial distribution of the clones within was achieved by overlaying a regular grid of squares on each half of the room and then placing clones randomly within a portion of the grids. This distributed the clones in a more realistic crowd-like manner than placing them on a completely regular grid. When created, each clone is turned toward the room's single exit and begins moving in that direction.

4.3. Evacuation scenarios

As mentioned, both panicked and nonpanicked evacuation scenarios were simulated. The methods used to implement the crowd members' movement differ between the scenarios. The implementation of the nonpanicked crowd uses both the Unity Physics engine and the Unity AI engine, the latter in the form of a Unity Navigation Mesh. The implementation of the panicked crowd uses only the Unity Physics engine; the crowd behavior is based entirely on custom scripts.

It is certainly possible that both the panicked and nonpanicked crowds and scenarios could have been implemented using a single integrated script. However, the "black box" nature of the Unity AI engine made such an implementation problematic, and thus our implementation consists of separate scripts for panicked and nonpanicked crowds. This implementation has the secondary benefit of simplifying the process of tuning the model's parameters for each scenario. Combining the panicked and nonpanicked modules is left for future work.

4.3.1. Nonpanicked evacuation

The instantiation of the scene (i.e., the room and exit) is similar in both the nonpanicked and panicked versions of the scenario, but there are some differences in the implementation of the crowd members. When the crowd is not panicked, the forces involving acceptable social distance discussed in Sec. 3 act on the clones, but the body forces and forces due to friction do not, because collisions typically do not occur. These physical forces are zero in the Helbing model. Therefore, the Crowd

Guy prefab for the nonpanicked scenario suggests a different configuration than for the panicked scenario.

One of the differences is the use of Unity Navigation by the nonpanicked Crowd Guy clones to plot a path through the exit while avoiding objects, or other clones, in their way. By causing the clones to avoid objects in their path, Unity Navigation provides the necessary functionality to simulate social forces in crowds.³⁴ In order to use Unity Navigation in the nonpanicked scenario, a Navigation Mesh, or NavMesh, is created. The NavMesh is a simplified geometrical structure that consists of all the stationary objects within the scene. A NavMesh component, or agent, is then added to each clone; it enables them to plot a path around the various obstacles in the NavMesh.

The nonpanicked scenario uses NavMesh to take advantage of the Unity AI engine. While NavMesh can be tuned to an extent, it does not allow for the level of granularity of the simulation results demonstrated in prior work.³³ However, the choice of the NavMesh simplified the implementation of the nonpanicked scenario significantly, and required only limited calibration to produce realistic results.

```

1  using UnityEngine;
2  using System.Collections;
3
4  public class MoveTo : MonoBehaviour {
5
6      Vector3 target;
7      static private int health;
8      private static bool dead = false;
9      private bool escaped = false;
10     private Spawn_Crowd_NP SC;
11
12     // Use this for initialization
13     void Start () {
14
15         Spawn_Crowd_NP SCObject =
16             (Spawn_Crowd_NP) FindObjectOfType (typeof(Spawn_Crowd_NP));
17         if (SCObject != null) {
18             SC = SCObject.GetComponent<Spawn_Crowd_NP> ();
19         }
20
21         if (SC == null) {
22             Debug.Log ("Cannot find 'Spawn_Crowd_NP' script");
23         }
24
25         target = new Vector3 (0, 0.5f, -300);

```

```

26
27     UnityEngine.AI.NavMeshAgent agent =
28         GetComponent<UnityEngine.AI.NavMeshAgent> ();
29     agent.destination = target;
30     health = 500;
31 }

```

Code excerpt 1. MoveTo method for nonpanicked scenario.

Once a destination was specified for the clone to move toward, a component script called `MoveTo` was added to the NavMesh agents. Code excerpt 1 is the `MoveTo` method from the nonpanicked scenario. Lines 6–10 declare and initialize the NavMesh agent. Lines 15–16 find the NavMesh agent of the instantiated prefab called `Spawn.Crowd_NP`. Lines 17–23 handle cases where the agent is not found. Lines 27–29 establish the destination and direct the clone to move toward the destination. The target destination is set to $(0, 0, -300)$, which is a large distance outside of the room. This was necessary so that the clones could exit without blocking the exit for other clones still trying to escape the room. Using the NavMesh ensures that the clone can find the target and will avoid obstacles and other clones. Line 30 initializes the health of the clone at 500 units.

As the simulation runs, another script, `Update`, is called once per frame. On each update, the location of the clone is evaluated. When the clone reaches $z = -50$, it is considered to have escaped from the room.

An event handler, `OnCollisionEnter`, which is called when a clone collides with another clone or object in the scene. `OnCollisionEnter` decrements the clone's health value. The clone is marked dead if its health is zero or less, and statistics are updated to reflect the count of dead clones. However, in the nonpanicked scenario no collisions were recorded, as expected.

Figure 4 shows an overhead view of an execution of the nonpanicked scenario. The crowd is forming the characteristic arch around the exit, as seen earlier in Fig. 1. Figure 5 shows the same nonpanicked scenario from a different angle. In doing so, the nonpanicked simulation therefore may be considered to satisfy a qualitative face validation.

4.3.2. Panicked evacuation

The implementation of the panicked scenario is different from the nonpanicked scenario in several ways. Notably, the clones move faster, and the movement is not facilitated by the Unity AI engine through the NavMesh agent, unlike the nonpanicked scenario.

As stated previously, when panic occurs, the social forces are added to the physical forces of collisions with other crowd members. The clones are scripted to move to a target $(0, 0, -49)$ which is just inside the room (see Fig. 2), then turn and move through the door, at which time, they are considered to have escaped.

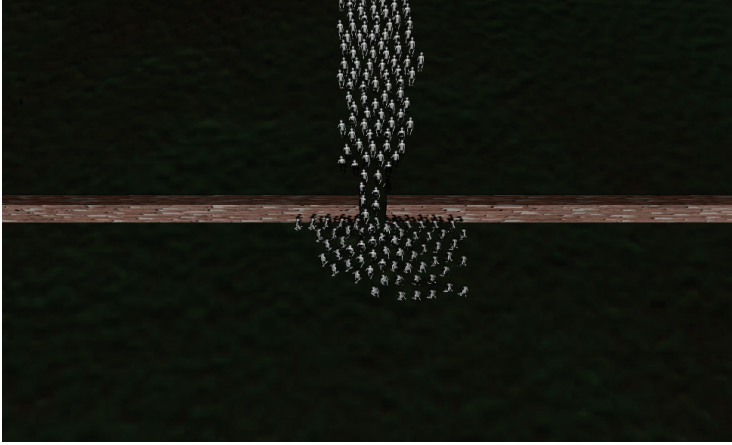


Fig. 4. Overhead view of the nonpanicked scenario.

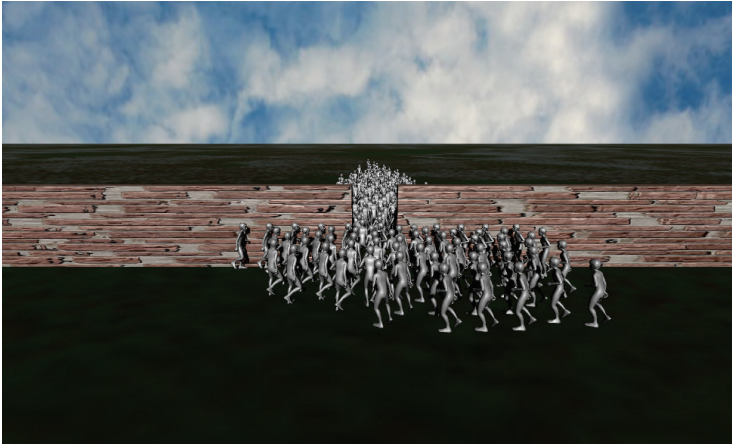


Fig. 5. Oblique view of the nonpanicked scenario.

Code excerpt 2 shows the **Update** method of the panicked scenario, which is executed every frame. The nonpanicked and panicked versions of **Update** differ in that they are attached to different classes of objects and the physical forces necessary to model crowd panic are implemented. In the panicked scenario, **Update** is a script component of the **CharacterController** object, while in the nonpanicked scenario, **Update** is a component of the **NavMesh** agent. Because the **NavMesh** agent provides appropriate social distancing demonstrated in nonpanicked situations, its absence in the panicked scenario more accurately models the panicked crowd.

Code excerpt 2 represents the portion of Eq. (1) related to motion in the desired direction relative to the current direction. Line 2 retrieves the object reference for

```
1 void Update () {
2     var cc = GetComponent<CharacterController> ();
3     if (escaped) {
4         cc.SimpleMove (direction * Time.deltaTime * speed);
5         return;
6     }
7
8     h = Mathf.Sqrt(Mathf.Pow(
9         (target.x - transform.position.x), 2) +
10        Mathf.Pow((target.z - transform.position.z), 2));
11
12    Vector3 relativePos = target - transform.position;
13    Quaternion rotation =
14        Quaternion.LookRotation (relativePos);
15    transform.rotation = rotation;
16
17
18    direction = new Vector3 ((target.x -
19        transform.position.x) / h, 0,
20        (target.z - transform.position.z) / h);
21
22    direction.x = direction.x - (dv.x / h);
23    direction.y = direction.y - (dv.y / h);
24    direction.z = direction.z - (dv.z / h);
25
26    if (!escaped & transform.position.z < -49.0f) {
27        escaped = true;
28        SC.addEsc ();
29    }
30
31    if(transform.position.z < -49.0f) {
32        direction.z = -1;
33        direction.x =
34            (int) (((float) Random.value - 0.5) * 2.0);
35        Vector3 v3 =
36            new Vector3 (direction.x, 0.0f, direction.z);
37        rotation = Quaternion.LookRotation (v3);
38        transform.rotation = rotation;
39    }
40}
```

```

41    cc.SimpleMove (direction * Time.deltaTime * speed);
42 }

```

Code excerpt 2. Update method for panicked scenario.

the CharacterController connected to the clone. Lines 3–6 test whether or not the clone has already escaped, and if so, continues movement in the current direction, and returns. If the clone has not escaped, lines 8–10 calculate the distance from the exit target, specified at $(0, 0, -49)$, to the current position of the clone. Lines 12–15 direct the character to turn toward the target. Lines 18–24 create a normalized direction vector toward the target. Lines 26–29 check whether the clone has escaped during the current update, and if so increment the escaped count. Lines 31–39 turn the clone to go through the doorway if $(z < -49)$, if it has been reached, and continue moving in that direction. A random element is used for the direction so that the clones don't clump together as they move beyond the target. Line 41 uses the `SimpleMove` method to move the clone toward the target and beyond. `SimpleMove` ignores the y component of the direction vector and moves the clone proportional to the speed variable.

In Code excerpt 3, the method `OnControllerColliderHit` is an event handler that is run whenever a CharacterController detects a collision. It implements the components of Eqs. (1) and (2) related to forces sustained in a collision with a person or an object, respectively. Line 2 finds the collider that collided with the clone and line 5 calls the method `takeDamage` to record the damage to the character the clone collided with. Lines 14–21 calculate the values of the 3-element vector dv which represents the difference in force applied to a person because of a collision. These calculations are consistent with the Helbing model for changing direction of crowd members based on collisions.

```

1  void OnControllerColliderHit(ControllerColliderHit hit) {
2      var CPS = hit.collider.GetComponentInChildren
          <CrowdPersonScript> ();
3      if (CPS == null)
4          return;
5      CPS.takeDamage (hit);
6      n_ij = hit.normal;
7      t_ij.x = n_ij.z;
8      t_ij.z = n_ij.x;
9      t_ij.y = 0.0f;
10     v_j = CPS.getDirection ();
11     int k = 5000;
12     int K = 3000;
13     Vector3 diff;
14     diff.x = v_j.x - direction.x;
15     diff.y = v_j.y - direction.y;

```

```
16     diff.z = v_j.z - direction.z;
17     dv_ji = Vector3.Dot (t_ij, diff);
18     f_ph = k * n_ij + K * dv_ji * t_ij;
19     dv.x = direction.x - transform.position.x - target.x + f_ph.x;
20     dv.y = direction.y - transform.position.y - target.y + f_ph.y;
21     dv.z = direction.z - transform.position.z - target.z + f_ph.z;
22 }
```

Code excerpt 3. OnControllerColliderHit method in panicked scenario.

In the method `takeDamage`, after verifying that the clone has not escaped, the clone's health value is decremented. If the character's health is negative, the character is determined to have died in the simulation, and is replaced by a `Ragdoll`, which falls to the ground. Counters are updated indicating number of collisions, deaths, and escapes.

Code excerpt 4 shows the `addColl` and `updateDisplay` methods. Lines 1–13 are the `addColl` method, which updates the collision count, and if necessary, the death count. If the clone has died, a new location vector is created equal to that of the dead game object passed as a parameter to `addColl`. On line 10, a new prefab is instantiated at the location of the dead character. This new character is a Unity object known as a `RagDoll`, which immediately falls to the ground. `RagDolls` contain `RigidBody` colliders with each portion of the rigged character and will collide with `CharacterControllers`, so they become obstacles in the scene so that remaining clones must bypass or climb over.³⁴

```
1  public void addColl(bool dead, GameObject go) {
2      collisions++;
3      if (dead) {
4          deaths++;
5
6          float x = go.transform.position.x;
7          float y = go.transform.position.y;
8          float z = go.transform.position.z;
9          Vector3 v3 = new Vector3 (x, y, z);
10         Instantiate (GR_RD, v3, Quaternion.FromToRotation
            (target, v3));
11     }
12     updateDisplay (dead);
13 }
14
15 void updateDisplay(bool dead) {
16     string s = "Time elapsed: " + currentTime + "\nCollisions: " +
        collisions + "\nDeaths: " + deaths + "\nEscaped: " +
        escaped;
```

```

17     if (collisions % 1 == 0) {
18         collText.text = s;
19         //writer.Write (currentTime);
20         //writer.Write (" - ");
21         writer.Write (collisions);
22         writer.Write (" - ");
23         writer.Write (deaths);
24         writer.Write (" - ");
25         writer.WriteLine (escaped);
26     }
27 }

```

Code excerpt 4. AddColl and updateDisplay methods in panicked scenario.

Figure 6 shows an overhead view of the panicked scenario. The crowd is in the process of forming a semicircular arch around the exit in a similar manner to the nonpanicked scenario. Unlike the nonpanicked version, the clones are very closely packed around each other, the walls, and the exit. It can be seen that although the clones are escaping, they are more physically dispersed beyond the exit than in the nonpanicked scenario (Fig. 4), because there are fewer escaping during each period of time. Figure 7 shows another run of the same scenario from another angle. This shows the dead clones on the ground and the live clones attempting to escape by climbing over them.

4.4. *Functionality testing*

The objectives of the functionality testing were to assess the correctness of the model implementation and to confirm qualitatively reasonable results that were

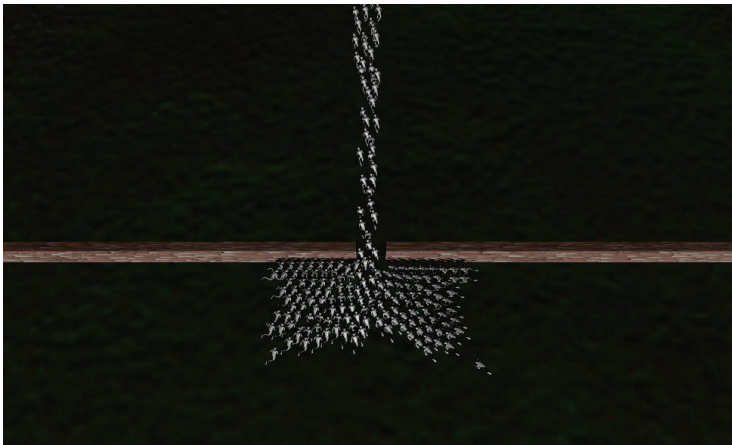


Fig. 6. Overhead view of the panicked scenario.

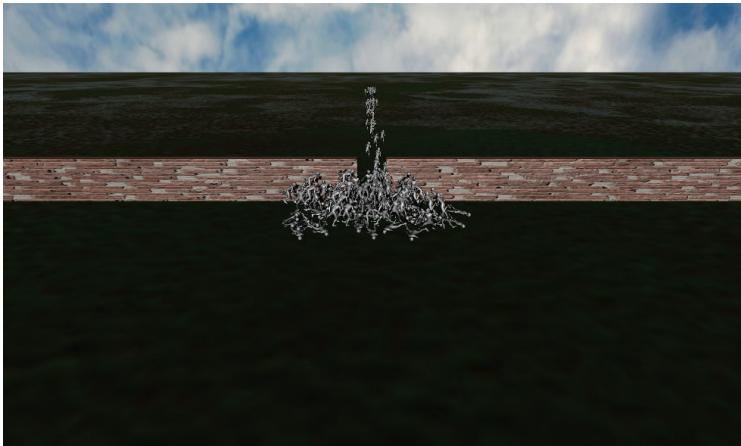


Fig. 7. Oblique view of the panicked scenario.

Table 1. Initial testing results; means are over 5 executions for each crowd size and state.

Crowd		Results (mean)			
Size	Scenario	Time	Collisions	Deaths	Escapes
50	Nonpanicked	31.2	0	0	50.0
	Panicked	31.7	2,155.6	0	50.0
100	Nonpanicked	40.3	0	0	100.0
	Panicked	34.6	29,601.8	27.4	71.6
150	Nonpanicked	42.3	0	0	150.0
	Panicked	39.4	46,385.8	51.2	100.2
200	Nonpanicked	45.1	0	0	200.0
	Panicked	64.6	65,387.8	69.2	130.4
250	Nonpanicked	49.8	0	0	250.0
	Panicked	61.7	102,094.2	102.2	144.8

consistent with expectations. The basic evacuation scene already described, which has a simple rectangular room with a single exit, was used for this testing. Five different crowd sizes were used (50, 100, 150, 200, and 250) and both scenarios, nonpanicked and panicked, were tested. The scene did not change between the different crowd sizes and scenarios. The simulation was executed five times for each combination of crowd size and scenario. Figures 5 and 6 are from an execution with a nonpanicked crowd of size 200, and Figs. 6 and 7 are from an execution with a panicked crowd of size 200. Table 1 summarizes the quantitative results of the simulations. The table shows the means of evacuation time, number of collisions, number of deaths, and number of escaped crowd members for each combination of crowd size and scenario. The evacuation time is the time when the last crowd member exited the room or was killed.

The nonpanicked scenario produced an orderly crowd moving toward the exit. The exit became congested, but the individual crowd members were able to pass through relatively efficiently, without causing any damage to each other. Because the crowd members are implemented using Unity Navigation, the nonpanicked crowd members maintained social distance, and thus there were no collisions or deaths in any of the nonpanicked executions for any crowd size.

In the panicked scenario the crowd members moved faster than in a nonpanicked state. As a result, the exit became clogged more quickly. Because of the panic and congestion, the crowd members began pushing each other while trying to reach and pass through the exit. This caused a number of collisions to occur, and several crowd members received enough damage to cause death. The numbers of collisions and deaths increased with the crowd size. The crowd members who were not killed had to climb over the dead crowd members, further slowing their egress. The rate of escape for the crowd members was irregular, which was consistent with other implementations of the Helbing model.³²

Both scenarios produced results consistent with our expectations, which were derived from other simulations using the Helbing model³³ and our earlier work.³² During the simulations the evacuating crowd members formed a realistic arch formation around the exit door, which constituted an informal qualitative face validation of the model.³⁶ These simulations also show that when the crowd does not panic, evacuation can occur without injury, which is consistent with the literature.¹

5. Validation

Once the functionality testing had been successfully completed, quantitative calibration and validation were performed based on an actual crowd evacuation event with a more complex scene. Validating a model by recreating and comparing its results to an actual event is an effective validation method known as predictive validation³⁷ or retroactive prediction (“retrodiction”); it is often (but certainly not exclusively) used for combat models.³⁸ This section reports the process and results of quantitatively validating the model.

5.1. Validation event

The event used for model validation was a panicked evacuation during a nightclub fire that occurred at The Station nightclub in West Warwick, Rhode Island on February 20, 2003.³⁹ This event was chosen for validating the model because of the quantity of data available, the detail available in that data, and the authoritative source of that data.³⁹ The same event has been used at least twice before in model validation, although in a different way each time.^{20,40}

In the actual event, pyrotechnics used at the start of a live concert inside the crowded nightclub ignited polyurethane foam insulation in the walls, starting a fire that quickly triggered a panicked evacuation. By coincidence, on the night of the fire a camera operator from a local television station was at the nightclub recording

W. A. Cantrell et al.

for a report on nightclub overcrowding; that report was motivated by a deadly stampede that had occurred at another nightclub in Chicago three days earlier.⁴¹ The camera operator captured the fire and evacuation on video from both inside and outside the nightclub because he was able to evacuate safely. That video and the subsequent investigation documented the evacuation timeline and events (such as fire ignition, specific individual exits, clogging near the front door, and responses by emergency personnel), estimates for the density of the crowd, and details regarding number of persons escaped and the exits they used to do so.

The total number of people inside the nightclub building when the fire started has been reported as 458³⁹ and 462.⁴² For this work, 458 was used. The fire caused a total of 100 deaths, 96 during the event and four later due to injuries sustained during the event. The deaths were attributed to both the fire itself, including burns and smoke inhalation, and to crushing forces produced by the panicked crowd. The press of the panicked crowd hindered the ability of some crowd members to exit safely before being overcome by the thick black smoke. Many people had less than 90 s from the time of initial ignition to escape. Panicked evacuation did not start until approximately 30 s after ignition; at first, some people briefly thought the fire was an intentional part of the concert's fireworks.³⁹

The building had four different doors; all of these doors, as well as two different windows were used to escape the building. However, one of those doors, the platform exit, was near the location of the initial ignition and was only accessible within the

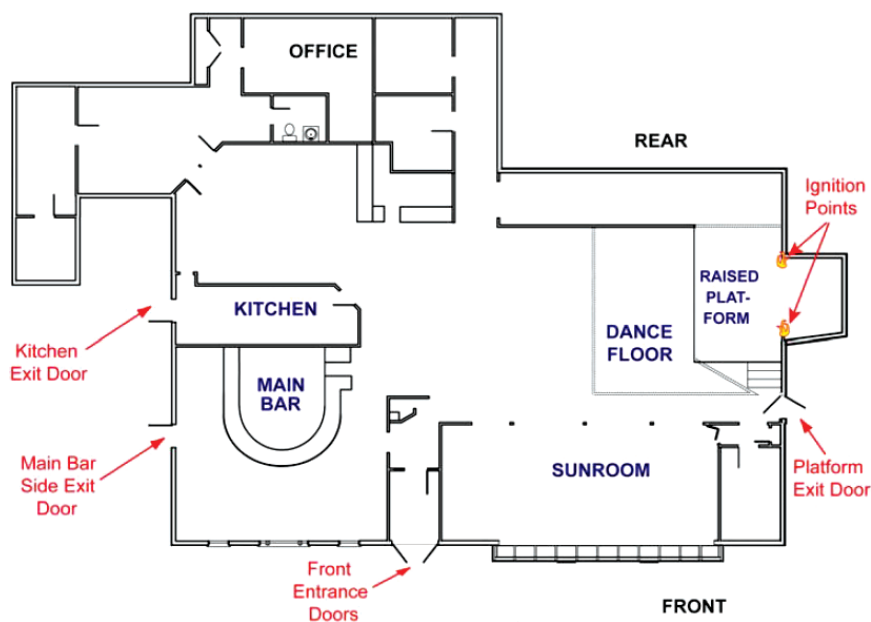


Fig. 8. Floorplan of The Station nightclub.⁴⁰ The main door is at the bottom.

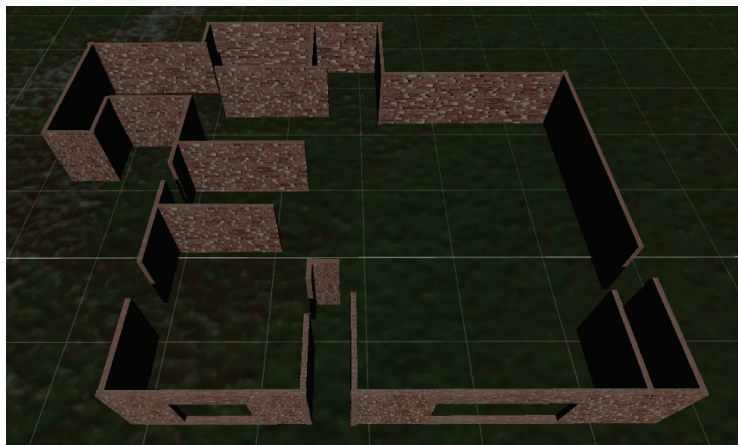


Fig. 9. Unity scene implementation of The Station nightclub, seen from the same direction as Fig. 8.

first 30s of the fire, and another of the doors, the kitchen exit, was not known to most of the people and thus it was not used by many to escape.⁴³

5.2. Scenario implementation

The Station nightclub building and the evacuation were recreated in Unity. Doing so required replicating the building layout, instantiating the correct number of crowd members at appropriate places within the building, and assigning an exit to each crowd member in a proportion consistent with the exits used during the actual evacuation.

Figure 8 shows the nightclub's floor plan and Fig. 9 shows the Unity implementation of it. Some of the walls and smaller rooms used for storage and office space were not implemented for simplicity; the omitted rooms were not on the evacuation routes and did not affect the crowd members' egress times.

For each simulation execution, spawn points for the crowd members are created within the nightclub's main room. The `Spawn_Crowd` script was modified to support generating new spawn points for the crowd members inside the nightclub walls. Two uniformly distributed random variates are generated within the x and z intervals representing the boundaries of the nightclub's main room. These random variates are then used as a Crowd Guy clone's initial position. The process is repeated for each crowd member.

In Code excerpt 5, each clone is randomly assigned one of the six exits, through which the clone will attempt to exit the nightclub. The exit assignment process does not represent an emergency responder managing the evacuation; rather it represents the individual exit selection decision made by each crowd member during the event. Research has shown that in a panicked evacuation crowd members tend to prefer

```

1  float ran;
2  if (transform.position.x >= -12.5)
3      //left half of nightclub
4      ran = Random.Range(0.0f, 0.69f);
5      //probabilities of main, platform doors, left window
6  else { //right half of nightclub
7      //don't assign to kitchen door if person is past kitchen
8      if (transform.position.z <= -42.5f)
9          ran = Random.Range (0.13f, 1.0f);
10         //probabilities of main, bar, kitchen doors, right window
11     else
12         ran = Random.Range (0.13f, 0.97f);
13 }
14
15 if(ran < 0.05)
16     tgt = new Vector3(16.5f, 0.5f, -51.5f); //platform exit door
17 else if(ran < 0.13)
18     tgt = new Vector3(0.25f, 0.5f, -63.5f); //left window - 0.08
19 else if(ran < 0.69)
20     tgt = new Vector3(-14.5f, 0.5f, -60.0f);
21     //main door - 0.56
22 else if(ran < 0.85)
23     tgt = new Vector3(-25.5f, 0.5f, -63.5f);
24     //right window - 0.16
25 else if(ran < 0.97)
26     tgt = new Vector3(-33.5f, 0.5f, -52.5f); //bar door - 0.12
27 else
28     tgt = new Vector3(-33.5f, 0.5f, -39.5f);
29     //kitchen door - 0.03
30
31 currentTgt = tgt;

```

Code excerpt 5. Nightclub scenario crowd member exit assignment.

to exit a building through the doorway they are most familiar with.^{44–46} In an unfamiliar building, that is most likely to be the same doorway through which they entered the building, which explains the relatively large probability associated with the main door.

The nightclub's main room is partitioned into two areas so that the clones would be assigned an exit near their initial location. Clones in the right area (with respect to Figs. 8 and 9) are randomly assigned to exit through either the platform exit door, the left window, or the main door. Clones in the left area are randomly assigned to exit through either the main door, right window, bar door, or kitchen

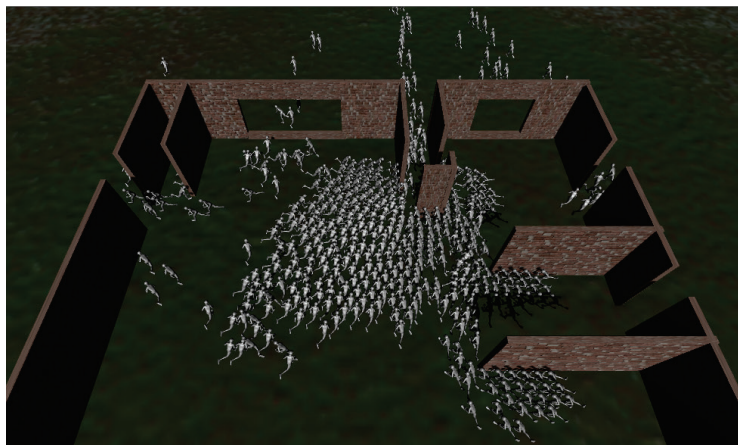


Fig. 10. Overhead view of an execution of the nightclub evacuation simulation. The nightclub's main door is at the top.

door. The probability of a given clone being assigned to a specific exit is proportional to the number of crowd members who exited via that door or window in the actual evacuation event. Because the assignments are stochastic, there is run-to-run variation in the exit assigned to each crowd member and the total number of crowd members assigned to each exit.

As the simulation executes the script checks to see if a clone has escaped, due to internal walls, the clones had to be routed to their assigned exit point from their initial spawn points. This was accomplished by creating intermediate target points for the clones based on their initial location.

Figure 10 shows an overhead view of an execution of the nightclub evacuation simulation. The evacuating clones have formed the characteristic semi-circular arch around the main exit in the manner expected. Some of the clones are exiting through other doors or windows according to their assigned exits. Figure 11 shows the same simulation, but a short time later, from another angle. Figure 12 shows the end of the simulation; dead clones are near the main exit door.

5.3. Calibration

Model calibration is the process of iteratively comparing the model and its results to the real-world system being modeled and revising the model to improve its accuracy.⁴⁷ When tested on the nightclub scenario, the model initially produced unrealistically high numbers of collisions and deaths. Calibration was required to bring the results of the model in line with the actual event.

The model was calibrated by iteratively adjusting the values of two of the clones' parameters: movement speed and initial health. The movement speed parameter determined how quickly the clones moved toward their assigned exit, unless slowed



Fig. 11. Oblique view of an execution of the nightclub evacuation simulation.

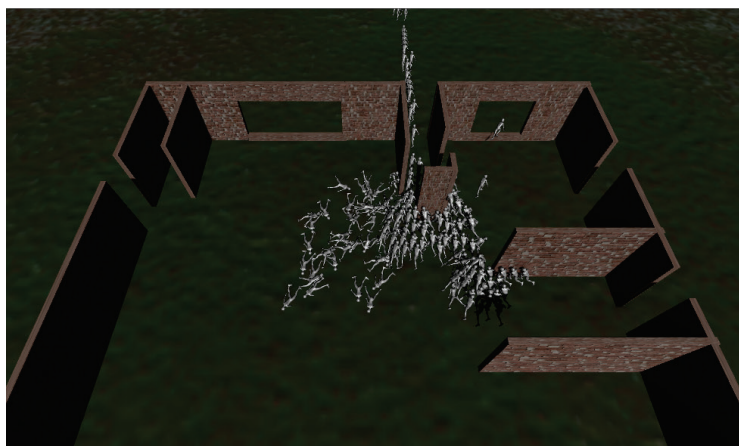


Fig. 12. Overhead view of the nightclub evacuation simulation after execution has ended.

by congestion. The health parameter determined how many collisions with other clones would result in a clone's death.

The model was executed and the parameter values adjusted to find values that both produced results consistent with the actual evacuation and were physically realistic. During the early part of the calibration process, the speed and health parameter values were the same for all clones. With consistent parameter values the number of deaths by crowding was unrealistically higher than the number of deaths by fire and smoke inhalation. Later in the calibration process, values for speed and health that varied from clone to clone were randomly generated using normal distributions. Using health parameter values that varied from one crowd member to the next resulted in comparable numbers for the number of deaths by

crowding and the number of deaths by smoke inhalation and was more realistic. The parameter values that produced results that best matched the actual event were a consistent speed and a normally distributed health.

Ultimately, the calibration adjustments to speed and health led to values that produced total deaths in a simulation to closely approximate the 100 deaths that occurred in the actual event. As mentioned earlier, the crowd members had about 90 s after ignition to escape before succumbing to the fire and smoke inside the nightclub and the actual evacuation did not start until approximately 30 s after the fire started.⁴⁰ The validation runs were started at the moment the evacuation began, thus during a simulation clones remaining in the building after 60 s were considered to have died due to fire and smoke inhalation.

5.4. Validation results

Once the best values for the speed, health, and exit assignment probabilities had been found during the calibration process, the model was then validated. Because of the randomness present in the model, including spawn point locations, health values, and exit assignments for each crowd member, there was run-to-run variation in the simulation results. Therefore, during the validation process the nightclub evacuation simulation was executed 30 times. The results of each run, including deaths by trampling, deaths by fire and smoke, total deaths, and total escapes were recorded. Importantly, the parameter values set during the calibration process were not subsequently adjusted; no changes were made to the model or its parameters during the validation process.

Table 2 details the results of the 30 validation executions. The “Crowding deaths” column shows the number of crowd members killed due to collisions with other crowd members, the “Fire and smoke deaths” column shows the number of crowd members remaining in the building after 60 s and thus assumed to have succumbed to the fire or smoke inhalation, and the “Total deaths” column is the sum of the two types of deaths. The “Escaped” column is the number of crowd members who were able to exit the building.

Confidence intervals were used to quantitatively assess the results of the validation runs. Confidence intervals are often used for model validation when the results of multiple runs of a stochastic model are compared to a single actual event,^{48,49} or conversely, when multiple measurements subject to uncertainty or measurement error of a physical phenomenon are compared to the results of a single run of a deterministic model.⁵⁰ The crowd movement model is stochastic and one event was used for validation, thus the former situation is applicable here. The response variables considered in the validation were Total deaths and Escaped. The model was executed 30 times and the response variables’ values recorded. In addition to mean, standard deviation, minimum, and maximum for those response variables, confidence intervals with a confidence level of 95% were calculated using the Student *t* distribution.⁵¹ Table 3 details the results of those calculations. As can be seen in

Table 2. Results of the 30 validation simulation executions.

Run	Crowding deaths	Fire and smoke deaths	Total deaths	Escaped
1	50	62	112	346
2	54	32	86	372
3	45	59	104	354
4	68	32	100	358
5	50	49	99	359
6	56	50	106	352
7	58	57	115	343
8	52	66	118	340
9	58	39	97	361
10	50	79	129	329
11	57	42	99	359
12	50	56	106	352
13	51	37	88	370
14	37	57	94	364
15	48	42	90	368
16	47	46	93	365
17	53	56	109	349
18	53	36	89	369
19	52	44	96	362
20	47	56	103	355
21	41	25	66	392
22	51	47	98	360
23	41	61	102	356
24	54	41	95	363
25	40	36	76	382
26	63	41	104	354
27	48	88	136	322
28	43	60	103	355
29	55	69	124	334
30	57	51	108	350

the table, the confidence interval for Total deaths is [96.11, 106.89], which includes the actual event value of 100, and the confidence interval for Escaped is [351.11, 361.89], which also includes the actual event value of 358. Therefore, the model is considered to be valid for those two response variables.^{48,49}

It could be argued that the two response variables, Total deaths and Escaped, are actually two complementary views of one response variable because Total deaths and Escaped always total to 458, the number of crowd members initially in the nightclub. From that perspective the model was validated for one response variable (albeit an important one), either Total deaths or Escaped, using the appropriate confidence interval.

6. Findings and Future Work

This paper has described in some detail how to implement a physics-based model of crowd movement in the Unity game engine. Unity contains a rendering engine

Table 3. Confidence interval calculations for the validation executions ($n = 30$).

Statistic	Total deaths	Escaped
Sample mean $\bar{x}$	101.50	356.50
Sample standard deviation s	14.44	14.44
Minimum	66	322
Maximum	136	392
Confidence interval degrees of freedom $d.f.$	29	29
Confidence level	95%	95%
Confidence interval critical value t_c	2.045	2.045
Confidence interval lower bound L	96.11	351.11
Confidence interval upper bound U	106.89	361.89

for 2D and 3D graphics generation and a physics engine which includes collision detection functionality. The relative ease of creating a usable 3D representation of real or imagined worlds in Unity compares favorably to level of effort likely to have been required to implement the same model and scenarios in a high-level programming language.

The modeling results achieved align with previous efforts to implement this type of model.³² Critical characteristics of crowd evacuation found in the literature, such as the formation of arches around exits, were properly replicated within the Unity-based model. Overall, the implementation demonstrates that in such configurations the Unity game engine can be used to model crowd movement in a reasonably simple yet usefully realistic manner, and thus may be a viable tool for researchers interested in modeling crowd evaluation.

Furthermore, the quantitative validation of the model using The Station nightclub event showed that Unity-based models of crowd evacuation can be validated as accurate, as least for building configurations similar to the nightclub (small buildings with a limited number of exits and a single point of hazard). They also demonstrate a validation process to use for such models.

There are a number of ways this work could be extended and enhanced:

- Execute simulations with increasing crowd sizes, to explore the computational limits of the game engine.
- Use the Helbing model not only to determine post-collision direction for a crowd member, as in the current implementation, but also use its calculation of crowd pressure to determine how damaging each collision is. Consider relative damage in updating crowd member health.
- Model nondirect movement, i.e., crowd members following paths to their assigned exit other than direct routes, due to obstacles, poor visibility, or congestion.
- Model crowd members changing their exit selection decision to another exit if they are blocked en route to their selected exit.
- Allow varying crowd member acceleration and speed parameters to reflect individual response to perceived crowd congestion and the cumulative damaging effect of multiple nonlethal collisions.

- Model each crowd member's individual transition from a nonpanicked to a panicked state based on what they could have sensed about the situation, leading to simulations where not all of the crowd members will panic and begin evacuating at the same moment.
- Combine the implementations of panicked and nonpanicked crowds into an integrated script.
- Validate the model using additional historical crowd evacuation events, including those dissimilar to the event used (larger buildings, more exits, and/or multiple points of hazard).

Conflicts of Interest

The authors declare that there are no conflicts of interest regarding the publication of this paper.

Acknowledgments

The authors thank Unity Technologies, developers of the Unity game engine, for allowing the use of Unity Personal for this research. Two anonymous reviewers of an earlier version of this paper provided insightful and useful feedback which helped to substantially improve the paper. Finally, the authors respectfully recognize the victims and their families of the actual event used in this work for validation purposes. Our dispassionate use of it in a simulation study in no way diminishes our personal awareness of the tragedy of the event.

References

1. Farkas I., Helbing D., Molnar P., Vicsek T., Simulation of pedestrian crowds in normal and evacuation situations, *Pedest. Evac. Dyn.* **21**(2):21–58, 2002.
2. Thalmann D., Musse S., *Crowd Simulation*, Springer-Verlag, London, 2007.
3. Pelechano N., Allbeck J., Badler N., *Virtual Crowds: Methods, Simulation, and Control*, Morgan and Claypool, San Rafael, 2008.
4. Loftin R., Petty M., McKenzie F., Gaskins R., Modeling crowd behavior for military simulation applications, in Rouse W., Boff K. (eds.), *Organizational Simulation*. John Wiley and Sons, New York, pp. 471–536, 2005.
5. McKenzie F. D., Petty M. D., Kruszewski P. A., Gaskins R. C., Nguyen Q.-A., Seevinck J., Weisel E. W., Integrating crowd-behavior modeling into military simulation using game technology, *Simul. Gaming* **39**(1):10–38, 2008.
6. O'Sullivan C., Cassell J., Vilhjálmsson H., Dingliana J., Dobbyn S., McNamee B., Peters C., Giang T., Levels of detail for crowds and groups, *Comput. Grap. Forum* **21**(4):733–741, 2002.
7. Reynolds W., Flocks, herds, and schools: A distributed behavioral model, *ACM SIGGRAPH, Proc. Fourteenth Annual Conf. Computer Graphics and Interactive Techniques*, Anaheim California USA, ACM, New York, pp. 27–31, 1987.
8. Fillippidis L., Galea E., Gwynne S., Lawrence P., Representing the influence of signage on evacuation behavior within an evacuation model, *J. Fire Prot. Eng.* **16**(1):37–73, 2006.

9. Bandini S., Manzoni S., Situated cellular agents for crowd simulation and visualization, *Cybern. Syst. Int. J.* **38**(7):729–753, 2007.
10. Brogan D., Hodgins J., Group behaviors for systems with significant dynamics, *Auton. Robots* **4**(1):137–153, 1997.
11. Pelechano N., Allbeck J., Badler N., Controlling individual agents in high-density crowd simulation, *ACM SIGGRAPH, Proc. 2007 ACM SIGGRAPH/Eurographics Symp. Computer Animation*, pp. 99–108, San Diego, Eurographics, Aire-la-Ville, 2007.
12. Brenner M., Wijermans A., Nussle T., de Boer B., Simulating and controlling civilian crowds in robocup rescue, <https://www.semanticscholar.org/paper/Simulating-and-Controlling-Civilian-Crowds-in-Brenner-Wijermans/891b8d457218cf6d81b49d328c1a2be5608b123c>, 2005.
13. Sornum K., Liang Y., Cai W., Low M., Zhou S., 3D visualization and animation of crowd simulation using a game engine, *GSTF, Proc. 2009 Computer Games, Multimedia, and Allied Technologies Conf.* GSTF, Singapore, 2009, doi:10.5176/978-981-08-3190-5_455.
14. Braun A., Musse S. R., de Oliveira L. P., Bodmann B. E., Modeling individual behaviors in crowd simulation, *16th Int. Conf. Computer Animation and Social Agents*, IEEE, New Brunswick, NJ, USA, pp. 143–148, 2003.
15. Papelis Y. E., Kady R. E., Bair L. J., Weisel E. W., Modeling of human behavior in crowds using a cognitive feedback approach, *SIMULATION: Trans. Soc. Model. Simul. Int.* **93**(7):567–578, 2017.
16. Helbing D., Farkas I. J., Molnár P., Vicsek T., Simulation of pedestrian crowds in normal and evaluation situations, in Schreckenberg M., Sharma S. D. (eds.), *Pedestrian and Evacuation Dynamics*, Springer, New York, 21–58, 2002.
17. Keating J. P., The myth of panic. *Fire J.* **76**(3):57–61, 1982.
18. Elliot D., Smith D., Football stadia disasters in the United Kingdom: Learning from tragedy? *Org. Environ.* **7**(3):205–229, 1993.
19. Jia X., Yue H., Tian X., Yin H., Simulation of pedestrian flow with evading and surpassing behavior in a walking passageway, *SIMULATION: Trans. Soc. Model. Simul. Int.* **93**(12):1013–1035, 2017.
20. Niu L., Song Y., A simulation model fusing space and agent for indoor dynamic fire evacuation analysis, *SIMULATION: Trans. Soc. Model. Simul. Int.* **92**(8):215–232, 2016.
21. Shendarkar A., Vasudevan K., Lee S., Son Y., Crowd simulation for emergency response using BDI agents based on immersive virtual reality, *Simul. Model. Pract. Theory* **16**(9):1415–1429, 2008.
22. Kinsey M. J., Galea E. R., Lawrence P. J., Modelling evacuation using escalators: A London underground dataset, in Weidmann U., Kirsch U., Schreckener M. (eds.), *Pedestrian and Evacuation Dynamics 2012*, Springer, Cham Switzerland, 385–399, 2014.
23. Ronchi E., Fahy R., Colonna P., Berloco N., Validation and calibration of the EXIT89 evacuation model for road tunnel evacuation applications, in Weidmann U., Kirsch U., Schreckener M. (eds.), *Pedestrian and Evacuation Dynamics 2012*, Springer, Cham Switzerland, pp. 543–550, 2014.
24. Hu M., A high-fidelity three-dimensional simulation method for evaluating passenger flow organization and facility layout at metro stations, *SIMULATION: Trans. Soc. Model. Simul. Int.* **93**(10):841–851, 2017.
25. Al-Habashna A., Wainer G., Modeling pedestrian behavior with Dell-DEVS: Theory and applications, *SIMULATION: Trans. Soc. Model. Simul. Int.* **92**(2):117–139, 2016.

26. Jafer S., Lawler R., Emergency crowd evacuation modeling and simulation framework with cellular discrete event systems, *SIMULATION: Trans. Soc. Model. Simul. Int.* **92**(8):795–817, 2016.
27. Yue H., Wang S., Jia X., Shao C., Simulation of pedestrian evacuation with blind herd mentality under adverse sight conditions, *SIMULATION: Trans. Soc. Model. Simul. Int.* **92**(6):491–506, 2016.
28. Bonabeau E., Dorigo M., Theraulaz G., *Swarm Intelligence: From Natural to Artificial Systems*, Oxford University Press, New York, 1999.
29. Banarjee S., Grosan C., Abraham A., Emotional ant based modeling of crowd dynamics, *Proc. Seventh Int. Symp. Symbolic and Numeric Algorithms for Scientific Computing*, IEEE Computer Society, Timisoara Romania, pp. 25–29, 2005.
30. Lee R., Hughes R., Minimisation of the risk of trampling in a crowd, *Math. Comp. Simul.* **74**(1):29–37, 2007.
31. Low M., Cai W., Zhou S., A federated agent-based crowd simulation architecture, in Zelinka I. (ed.), *Proceedings of the 2007 European Conference on Modeling and Simulation*, Prague Czech Republic, Digitaldruck Pirrot, Dudweiler, pp. 188–194, 2007.
32. Bott M., Petty M., Implementing a physics based model of crowd movement using unreal development kit, *J. Gaming Virtual Worlds* **6**(3):275–296, 2014.
33. Helbing D., Farkas I., Vicsek T., Simulating dynamical features of escape panic, *Nature* **407**:487–490, 2000.
34. Unity Technologies, *Unity Manual*, <https://docs.unity3d.com/Manual/index.html>, 2016.
35. Vu Studios, *Relaxed Man Character*, <https://www.assetstore.unity3d.com/en/#!/content/32665>, 2015.
36. Petty M. D., Verification, validation, and accreditation, in Sokolowski J. M., Banks C. M., *Modeling and Simulation Fundamentals: Theoretical Underpinnings and Practical Domains*, John Wiley and Sons, Inc., Hoboken, NJ, pp. 325–372, 2010.
37. Balci O., Verification, validation, and testing, in Banks, J. (ed.), *Handbook of Simulation: Principles, Methodology, Advances, Applications, and Practice*, John Wiley and Sons, New York, pp. 335–393, 1998.
38. Barbosa S. E., Petty M. D., A survey and comparison of past instances of combat model validation by retrodiction, *Proc. Spring 2010 Simulation Interoperability Workshop*, SISO (Simulation Interoperability Standards Organization), Orlando FL, April 12–16, 2010.
39. Grosshandler W., Bryner N., Madrzykowski D., Kuntz K., *Report of the Technical Investigation of The Station Nightclub Fire*, Vol. 1, National Institute of Standards and Technology, Washington, DC, 2005.
40. Pan X., *Computational Modeling of Human and Social Behaviors for Emergency Egress Analysis*, Ph.D. Dissertation, Stanford University, June 2006.
41. Wilgoren J., 21 Die in Stampede of 1,500 at Chicago Nightclub, *New York Times*, February 18, 2003, Online at <http://www.nytimes.com/2003/02/18/us/21-die-in-stampede-of-1500-at-chicago-nightclub.html>, Accessed November 8, 2016.
42. Parker P. E., Tally of a tragedy: 462 were in the station on night of fire, *Providence J.* 2007, Online at http://res.providencejournal.com/hercules/extra/2003/station-fire/content/STATION_FIRE_LIST_12-03-07-QL81OLD_v55.2a82be5.html, Accessed November 10, 2016.
43. Bryner N., Madrzykowski D., Grosshandler W., Reconstructing the station nightclub fire: Computer modeling of the fire growth and spread, *Interflam 2007 (Interflam '07) International Interflam Conf. 11th Proc.*, NIST (National Institute of Standards and Technology), September 3–5, 2, pp. 1181–1192, 2007.

44. Graham T. L., Roberts D. J., Qualitative overview of some important factors affecting the egress of people in hotel fires, *Int. J. Hospitality Manage.* **19**(1):79–87, 2000.
45. Alp N. C., Çağdaş G., Occupants emergency behavior in Turkey, in Weidmann U., Kirsch U., Schreckenberg M. (eds.), *Pedestrian and Evacuation Dynamics 2012*, Springer, Cham Switzerland, pp. 1123–1133, 2012.
46. Sagun A., Anumba C. J., Bouchlaghem D., Designing buildings to cope with emergencies: Findings from case studies on exit preferences, *Buildings* **3**(2):442–461, 2013, doi:10.3390/buildings3020442.
47. Banks J., Carson J. S., Nelson B. L., Nicol D. M., *Discrete-Event System Simulation, Fifth Edition*, Prentice Hall, Upper Saddle River NJ, 2010.
48. Petty M. D., Calculating and using confidence intervals for model validation, *Proc. Fall 2012 Simulation Interoperability Workshop*, SISO (Simulation Interoperability Standards Organization), Orlando FL, pp. 37–45, September 10–14, 2012.
49. Petty M. D., Advanced topics in calculating and using confidence intervals for model validation, *Proc. Spring 2013 Simulation Interoperability Workshop*, SISO (Simulation Interoperability Standards Organization), San Diego, CA, pp. 194–204, April 8–12, 2013.
50. Oberkampf W. L., Roy C. J., *Verification and Validation in Scientific Computing*, Cambridge University Press, Cambridge, UK, 2010.
51. Brase C. H., Brase C. P., *Understandable Statistics: Concepts and Methods*, 11th edn., Cengage Learning, Stamford CT, 2015.